Sender I/O: A Constructed Comparison

Document Number:	D4053R1
Date:	2026-03-13
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com Steve Gerbino steve@gerbino.co

Table of Contents

Abstract

Two sender-based TCP echo servers are constructed from the C++26 specification - one routing I/O results through `set_value`, one through `set_error` - and compared against a coroutine-native echo server (Corosio[3]). The `set_value` approach matches coroutine-native ergonomics but bypasses the channel-based composition algebra. The `set_error` approach retains composition but converts every routine network event

into an exception. Neither construction achieves both. The authors invite any reader to construct a sender-based echo server that preserves compound I/O results, retains channel-based composition, and avoids exception round-trips, using C++26 facilities. The authors will incorporate any such construction into a future revision and re-evaluate every finding. This paper is informational and proposes no action.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The authors developed Corosio[3] and Capy[4] and have an interest in this space. This paper proposes no action. It constructs the best sender-based echo server the authors can build from the C++26 specification, places it next to a coroutine-native echo server, and shows what each approach costs. The authors invite improvements to the sender construction (Section 8).

---

2. The Coroutine-Native Echo Server

Corosio[3] `do_session`:

```
capy::task<> do_session()
{
    for (;;)
    {
        auto [ec, n] = co_await sock_.read_some(
            capy::mutable_buffer(
                buf_, sizeof buf_));

        auto [wec, wn] = co_await capy::write(
            sock_, capy::const_buffer(buf_, n));

        if (wec || ec)
            break;
    }
    sock_.close();
}
```

Each `co_await` yields `(error_code, size_t)` as a structured binding. The error code is a value. No exceptions. Both values are visible at the call site.

---

3. Why I/O Results Are Different

Infrastructure operations have binary outcomes:

Operation	Success	Failure
`malloc`	Block returned	Allocation failed
`fopen`	File handle returned	Open failed
`pthread_create`	Thread running	Creation failed
GPU kernel launch	Kernel running	Launch failed
Timer arm	Timer armed	Resource limit

Every row is binary. The three channels - `set_value`, `set_error`, `set_stopped` - map unambiguously. The three-channel model is correct for this class. `std::execution` serves it well.

I/O operations return compound results:

Operation	Result
`read`	`(status, bytes_transferred)`
`write`	`(status, bytes_written)`

Status and data, always paired. Both values are always present. Every OS delivers them together. io_uring carries `res` and `flags` in one CQE. IOCP returns a `BOOL` and `lpNumberOfBytesTransferred` in one call. POSIX `read()` returns `ssize_t` with `errno` set.

Chris Kohlhoff identified this in P2430R0[5] ("Partial success scenarios with P2300," 2021):

> *"Due to the limitations of the `set_error` channel (which has a single 'error' argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel."*

The remaining sections construct both approaches to routing compound results through three channels and show what each costs.

---

4. Constructing the Sender Echo Server

Two approaches exist for routing I/O results through the three-channel model. We construct the best version of each from the C++26 specification (P2300R10[1], P3552R3[2]).

---

5. Approach A: Route Through `set_value`

The I/O sender calls `set_value(error_code, size_t)` for all outcomes. Completion signature: `set_value_t(std::error_code, std::size_t)`.

P3552R3[2] Section 4.4 specifies that `co_await` on a sender with multiple value arguments yields a `std::tuple`. Structured bindings work:

```
auto do_session(auto& sock, auto& buf)
    -> std::execution::task<void>
{
    for (;;)
    {
        auto [ec, n] = co_await async_read(
            sock, net::buffer(buf));

        auto [wec, wn] = co_await async_write(
            sock, net::const_buffer(buf, n));

        if (wec || ec)
            break;
    }
}
```

The echo session is nearly identical to Corosio. The error code is a value. No exceptions. Both values are visible at the call site.

What Approach A Loses

The I/O sender never calls `set_error` for I/O results. The channel-based composition algebra is bypassed:

- **`when_all`** cancels sibling operations when one completes with `set_error` or `set_stopped` ([exec.when.all]). An I/O failure arriving through `set_value` looks like success to `when_all`. Siblings continue.
- **`upon_error`** intercepts `set_error` ([exec.then]). It is unreachable for these senders.
- **`retry`** (stdexec, not in the C++26 working draft) intercepts `set_error` and restarts the operation. An I/O failure arriving through `set_value` does not trigger it.

The three-channel model reduces to one channel with a structured result. `set_error` serves no purpose for I/O senders under this approach. (`set_stopped` retains its cancellation role.)

---

6. Approach B: Route Through `set_error`

The I/O sender calls `set_value(size_t)` on success and `set_error(error_code)` on any non-zero status. Completion signatures: `set_value_t(std::size_t)` and `set_error_t(std::error_code)` .

P3552R3[2] specifies that `set_error` is converted to an exception via `AS-EXCEPT-PTR` ([exec.general] p8). For `error_code`, this is `make_exception_ptr(system_error(ec))`, rethrown in `await_resume`. There is no non-throwing path:

```
auto do_session(auto& sock, auto& buf)
    -> std::execution::task<void>
{
    try {
        for (;;)
        {
            auto n = co_await async_read(
                sock, net::buffer(buf));

            co_await async_write(
                sock, net::const_buffer(buf, n));
        }
    } catch (std::system_error const& e) {
        // ECONNRESET, EPIPE, EOF arrive here
    }
}
```

What Approach B Loses

- **Information.** The byte count is discarded on any error. A `write` that sends 500 of 1,000 bytes before `ECONNRESET` produces `set_error(ECONNRESET)`. The 500 is gone.
- **Non-throwing error handling.** Every `ECONNRESET` - a routine network event - is `make_exception_ptr` plus `rethrow_exception`. Two heap allocations and a table lookup per I/O error.
- **Visible error path.** The error hides in a `catch` block separated from the `co_await` site. The programmer cannot inspect the error code and the byte count at the same call site.

---

7. The Trade-Off

Each sender approach pays a cost the coroutine-native approach does not.

Property	Approach A (`set_value`)	Approach B (`set_error`)	Corosio
Error code at call site	Yes	No (in `catch`)	Yes
Byte count at call site	Yes	No (discarded on error)	Yes
Partial write preserved	Yes	No	Yes
`when_all` cancels on I/O error	No	Yes	N/A
`upon_error` reachable	No	Yes	N/A
`retry` fires on I/O error	No	Yes	N/A
Exception on `ECONNRESET`	No	Yes	No
Channels used for I/O	1 of 3	3 of 3	N/A

Infrastructure operations (Section 3) face no such trade-off. Their outcomes are binary. The three channels map unambiguously.

---

8. Invitation

The authors invite any reader to construct a sender-based echo server that:

- preserves compound I/O results (error code and byte count visible at the call site),
- retains channel-based composition (`when_all`, `upon_error` fire on I/O errors),
- avoids exception round-trips for routine error codes, and
- uses only facilities in C++26 or in-progress proposals (P3552R3[2], P3149[6]).

The authors will incorporate any such construction into a future revision and re-evaluate every finding in this paper. If the sender approach, when done correctly, matches the coroutine-native ergonomics, this paper will say so.

References

1. P2300R10 - "std::execution" (Michał Dominiak et al., 2024). https://wg21.link/p2300r10

2. P3552R3 - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3

3. cppalliance/corosio - Coroutine-native networking library. Commit ce1c43e. https://github.com/cppalliance/corosio/tree/ce1c43e623fb7b0e198ffac52be9267eccf04ecb

4. cppalliance/capy - Coroutine I/O primitives library. https://github.com/cppalliance/capy

5. P2430R0 - "Partial success scenarios with P2300" (Chris Kohlhoff, 2021). https://wg21.link/p2430r0

6. P3149 - "async_scope - Creating scopes for non-sequential concurrency" (Ian Petersen, Jessica Wong, Kirk Shoop, et al., 2024). https://wg21.link/p3149

7. D4054R0 - "Two Error Models" (Vinnie Falco, 2026). https://wg21.link/d4054r0

8. IEEE Std 1003.1 - POSIX `read()` / `write()` specification. https://pubs.opengroup.org/onlinepubs/9799919799/

9. P4007R0 - "Senders and Coroutines" (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4007r0